

A From-Scratch CUDA CNN

Forward-pass inference for LeNet-MNIST, no framework at runtime

Ashish Dasu

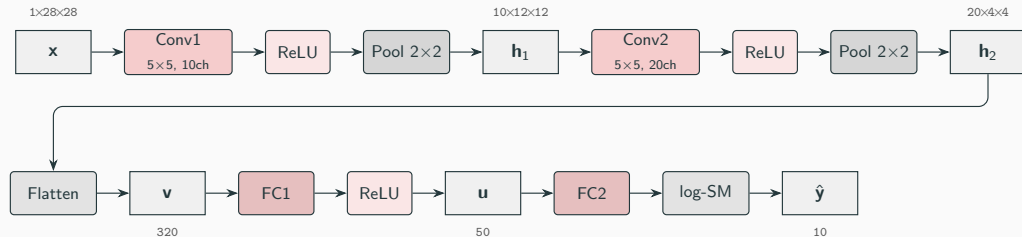
April 2026

CS5330 Pattern Recognition and Computer Vision

Khoury College of Computer Sciences, Northeastern University

Prof. Bruce Maxwell

The network and how we verified it

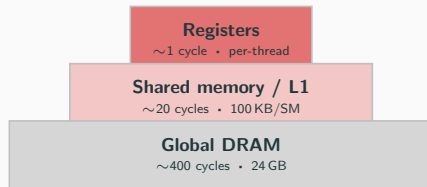
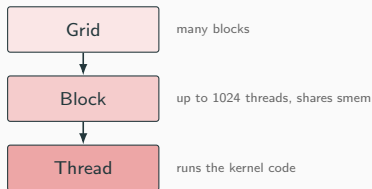


Correctness: CPU ref $\rightarrow$ per-kernel diff (10^{-4} tol) $\rightarrow$ 10k MNIST end-to-end. Max err $1.91 \times 10^{-5} \cdot \text{argmax}$
100% match $\cdot$ accuracy **97.93%**.

Background: a CUDA kernel, in one slide

Parallelism model. A kernel launch is one program executed by many threads at once.

Memory hierarchy. Latency costs dominate kernel design.



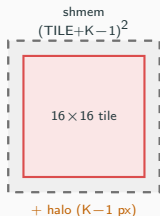
Threads in the same block can cooperate via `__shared__` memory and `__syncthreads()`; threads across blocks cannot.

The whole talk is one idea: move data from slow DRAM into fast shared memory / registers, reuse it, write results back.

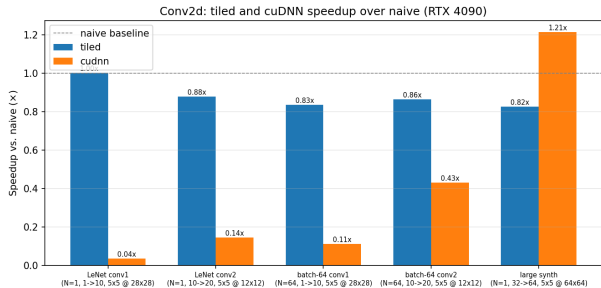
Opt. 1 — tiled shared-memory convolution

One block → one 16×16 output tile.

Threads cooperatively load input + halo into `__shared__` memory per channel; read shmem instead of DRAM.



Reuse factor = $K_h \cdot K_w = 25$. At LeNet scale the working set already fits in L1 — **naive wins**. Same tile strategy wins **6.4x** on a large GEMM (next slide). The *regime* decides.



Per-shape bars, RTX 4090, 200 reps after warmup.

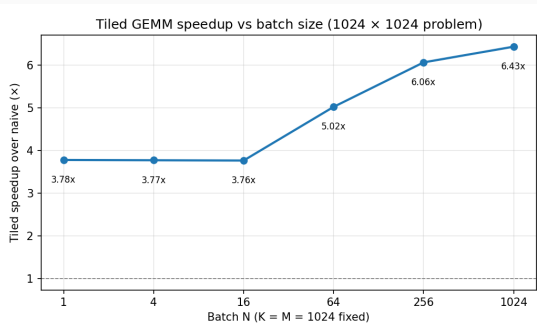
Tiled GEMM: where tiling actually wins

Same tiling idea applied to the fully-connected layers (matrix multiply Wx).

Plot: fixed 1024×1024 problem, sweep batch size N .

- $N=1$: **$3.8\times$** (tile already reuses weights)
- $N=64$: **$5.0\times$**
- $N=256$: **$6.1\times$**
- $N=1024$: **$6.4\times$**

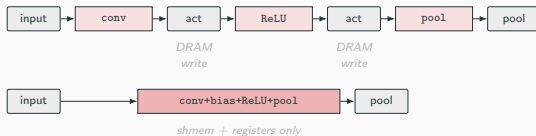
At LeNet's $fc1$ (50×320) and $fc2$ (10×50), the working set is tiny again — tiled is only $1.06\times$ / $1.01\times$ naive. Same *regime* story: tiling pays when there's enough data to amortize the shmem load.



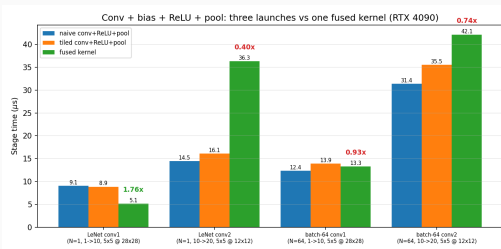
Tiled GEMM speedup vs batch size at $K=N=1024$.

Opt. 3 — conv + bias + ReLU + pool fused

Three back-to-back kernels at LeNet shapes spend more time on **launch overhead and DRAM round-trips** than arithmetic.



Each thread owns 4 conv accumulators in registers → ReLU in place → 2×2 max → one DRAM write.



conv1: naive 1.76× slower (launch overhead bound).

conv2: fused 0.40× naive — only 320 threads on 128 SMs, GPU starved.

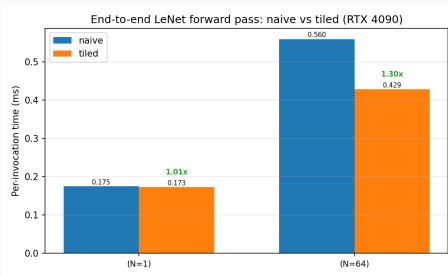
Fusion is a latency trade, not a free win.

Resource usage and cuDNN reference

Static analysis via `nvcc -Xptxas=-v` (Nsight counters blocked on pod, unprivileged container).

Kernel	Block	Regs	Smem
conv2d_naive	16×16	39	0
conv2d_tiled	16×16	40	1600 B
conv_relu_pool_fused	4×4	39	576 B
linear_naive	256	38	0
linear_tiled	16×16	37	2048 B
max_pool2d	16×16	20	0
relu_naive	256	8	0
log_softmax	16	15	64 B

From `nvcc -Xptxas=-v`. No register spills. Naive-vs-tiled gap is a **memory-access-pattern** story, not resource usage. `log_softmax` block=16 (half-warp) limits occupancy but costs only $\sim 1.8 \mu\text{s}/\text{call}$.



Naive beats cuDNN at batch 1 (descriptor overhead). cuDNN defaults to TF32 on Ada: $\text{err } 5.3 \times 10^{-3}$, $50\times$ our tolerance. Fixed by pinning CUDNN_FMA_MATH.

Demo: it actually works

Single-image inference (RTX 4090):

```
$ build/predict --bulk 42 --variant tiled
[predict] inference: 0.0312 ms
[predict] top-3 (log-prob):
  class 7 : -0.0001
  class 2 : -9.4823
  class 1 : -11.2144
[predict] argmax = 7 (label = 7) OK
```

Correctness: 11 / 11 tests pass.

- CPU reference (hand-computed)
- conv2d: naive, tiled, cuDNN, fused
- max_pool2d, ReLU, linear (naive & tiled), log_softmax
- end-to-end forward pass

Plus the 10 000-image MNIST parity harness: max abs err 1.91×10^{-5} , argmax-match on every image, accuracy 97.93%.

Discussion and reflection

- **Tiling isn't default-on.** Same tile strategy hurts at LeNet, wins $6.4\times$ on large GEMM. The regime decides.
- **Launch overhead is real.** Fusion buys $1.76\times$ on conv1; starves SMs on conv2. Same kernel, opposite outcome depending on grid size.
- **FP32 $\neq$ cuDNN default.** TF32 on Ada breaks 10^{-4} silently. Pin CUDNN_FMA_MATH explicitly.
- **Oracle chain paid off.** 11/11 per-kernel tests + 10k end-to-end parity — no “logits drifted” debugging.

“The optimization” and “the speedup” are separate things.

Thank you.

github.com/ashishdasu/cuda-cnn